Mini-Game-Hub Latex Report

Pravar Kumar and Shantanu Patil

29th April 2026

Introduction

This LaTeX report is a a document providing the insights on the list of external libraries used and their purpose by us throughout the project. Moreover it will along with including the features across the whole project will also give an explanation of the code logic for the generic class, each game, the authentication system, and the leader board. Even the hurdles we came across shall be discussed along with the external sources used by us. Lastly there would be a section dedicated to future prospects if e were given a course of 10 hr of additional time.

How to install and setup :

1. Install git:
   MAC : brew install git
   LINUX : sudo apt update and then sudo apt install git
   WINDOWS : Go to the official Git website: https://git-scm.com and run the .exe file

2. Install python:
   MAC : brew install python
   LINUX : sudo apt update and then sudo apt install python3 python3-pip
   WINDOWS: Go to https://python.org and install Python3 installer.

3. pip install pygame numpy matplotlib

4. install gawk by pip3 install gawk

5. install bash environment if you don't have MAC OS like us.

6. type git clone https://github.com/pravarkumar/mini-game-hub.git

7. type cd mini-game-hub

8. type bash main.sh
9. For further requirments type : vim requirements.txt
10. Finally most important part : Enjoy your game :)

External libraries used by us

The following are the set of external libraries used by us along with their use. They were used throughout the project though mainly in our game.py i.e. the menu file as we like to call it :

Library	**Purpose**
`pygame`	Used for GUI creation, animations, user input, and game rendering.
`sys`	Used for system-specific operations such as exiting the program.
`csv`	Used for reading and writing leader board or user data.
`os`	Used for file handling and operating system interactions.
`subprocess`	Used for executing external programs or processes.
`numpy`	Used for numerical operations and array handling.
`games.tictactoe`	Imports the Tic Tac Toe game class.
`games.othello`	Imports the Othello game class.
`games.connect4`	Imports the Connect 4 game class.
`matplotlib`	Helps us visualize the data and plot the graphs from the data
`collections`	Helps in easier torage of data as an associative array

Table 1: Libraries Used in the Project + Their purpose

Hurdles faced and how we overcame them

Even though the project was a very fun activity and we hardly hated doing it at any point still here goes nothing :

Hurdles	**Solutions**
The level of theory which we had for certain topics did not suffice for the needs of our imagination for the project interface.	We looked through numerous sources such as manuals, documentations, and many websites dedicated to those particular topics.
We were at times overwhelmed by such a large number of files and documents in our GitHub that keeping them in separate folders in an order way became a great hassle .	We had regular checks on our file system and thus were able to maintain a somewhat balanced file structure which was expected out of us :) .
Finding the appropriate references for our specific needs for the project also took a toll on us.	After searching through the web across many resources and also getting references from Claude we were finally able to find the appropriate resources as per our requirements.
We used up a lot of time in introducing the music option in our project. Our initial plan was to make the project such that across menu,Connect 4,Othello and even Ticktack Toe their specific background music would be playing but unfortunately we were informed that the lab systems didn't have the sound feature on them. We still kept the feature just in case.	We got to learn about how to introduce the music option which was an intriguing and an interesting activity .

Table 2: Hurdles faced during the project and their solutions

What would we do differently if we were given additional 10 hours

If we were given additional 10 hours then we would like to perform the follows :

- Make the project more aesthetic like addign more formatting and better graphics , probably also adding more features to each of the games.
- One thing which we would really love to do would be to make one more game (we were thinking of making the Flappy Bird game also but due to time constraints and many other unavoidable reasons we couldn't make the Flappy Bird game in time).
- Another point we would love to do would be to be able to introduce AI in the games as an opponent using certain algorithms wrt each game separately.

Bibliography with references and links to external sources

Following are the set of references which even though we didn't necessary used our work was inspired, references or based on these :

1. Python Documentation – https://docs.python.org/3/
2. Claude Ai– Image creation (mainly) and some simple coding difficulties
3. Pygame Learning + Overview – https://www.youtube.com/watch?v=jO6qQDNa2UYt=291s
4. Pygame Manual – https://www.pygame.org/docs/
5. Pygame Manual – pyga.me
6. Sample modular project – github.com/rennaMAhcuS/Maze
7. Pygame Manual – https://www.pygame.org/docs/
8. Overflow.com – https://www.overleaf.com/learn
9. Python inheritance – docs.python.org/3/tutorial/classes.html
10. Shell Functions – https://www.shellscript.sh/functions.html
11. GeeksforGeeks Python Tutorials – https://www.geeksforgeeks.org/python-programming-language/

Code Manual for our project :

1 main.sh (The Authentication of players)

This is the first file which we made this file is the first page which can be seen as a login to our mini-hub-game. Inside of main.sh we first defined different color and functions which we would use later in the terminal opening interface.

The ANSI escape codes we used were:

- Bold = “33[1m”
- Dim = “33[2m”
- Blink = “33[5m”
- Reset = “33[0m”

The main colors picked out by us included:

- Neon Cyan = “33[1;96m”
- Neon Pink = “33[1;95m”
- Neon Green = “33[1;92m”
- Neon Yellow = “33[1;93m”
- Neon Blue = “33[1;94m”
- Neon Red = “33[1;91m”
- White = “33[1;97m”
- Gray = “33[0;90m”

The functions we are about to showcase are the set of functions were used throughout the file which will help in the later understand of the logic . The main logic of each function is commented inside main.sh please refer to that for the logic ,as of the usage of functions it is as follows:

Shell Functions ()	**Usage**
sleep_fast	It is an extension of the inbuilt sleep function like if we input some value the program sleeps for that much time we set the default value as 0.05 sec for this.
center_text	As the name suggests the functions accepts the text and centers it using padding of appropriate amount. We have assumed the width of the terminal to be 80 in this case but otherwise also it should be more or less fine
divider	It is a simple function which just draws a horizontal line to separate different stages of login and other important distinctions.
typewrite	It is a function which makes the text inputted appear with delay creating a beautiful effect of as if someone is manually writing the text at that time.
boot_sequence	This is the main starting point of the main.sh file. The program first calls the boot sequence which will ultimately call other functions as per its need. This function can also be thought of the main interface display-er.
section_header	Displays the main heading of the section
fancy_prompt	This is yet another simple function which just makes the text appear more aesthetic and presentable.
secret_prompt	This is the function to input secret data such as password.
registered	returns 0 if we were able to register and 1 if we were which will help to see which loop to iterate next.
auth_player	This function just the main logic of the authorization of the players.
match_intro	This function is the extro message in a nutshell showing the names the online players and starting the game.

Table 3: Shell Functions used throughout the project

2 Workflow of main.sh (The authentication system

main.sh is the login page. It starts with the Mini-game-hub logo followed by getting credentials from player 1 first followed by player 2. The main authentication logic is as follows:

- Firstly player 1's inputted username is checked for validity, i.e. doe sit have valid characters or not. The user is even informed if there is an issue with the username they inputted.

- Then player 1 is constantly asked for username until they either have the username already registered, i.e. the username they inputted is present as the one of the first entries in the user.tsv or they are willing to add the input as a new username. We extract the first column by the "cut" command after matching the username by the "grep" command and extracting the row.

- After that, the password inputted is converted by hashing and compared to the password stores int he user.tsv corresponding to the username. The lop continues if the passwords don't match and terminates otherwise giving the appropriate feedback in either case.

- The same procedure is followed for player 2 and the game continues after both the user have been authorized.

- Finally, the ending screen and the loading prompt is shown to both the users indicating the start of the game.

3 Generic class logic :

The Generic class is the parent class for all the three games which we have made i.e. Connect 4 , TickTackToe and Othello. All the games thus are child classes of this class and by the concept of inheritance will be able to access all the methods of the parent i.e. the generic class (Game).These methods defined by us were as follows:

Methods	**Usage**
__init__	This is a dunder method used to initialize any game's players and the board with respect to that particular game.
switch	This is a simple method which is used to change the turn as the games work on turn basis logic where each player needs to switch the turn thus this function becomes significant.
check_win	This is an interesting one. Here as all the games have their own unique way of checking their check win condition os here we will it just as an error thrower. The method made in the child class will overwrite this method but if we forget to make it in the child class then this method will throw an error thus being a handy function.

Table 4: Methods used in the generic class

```
# Base Game Class
class Game:
    def __init__(self, current_player, player1, player2, row, col):
        self.player1 = player1
        self.player2 = player2
        self.board = np.zeros((row, col))
        self.current_player = player1

    def switch(self):
        self.current_player = self.player2 if self.current_player == self.player1 else self.player1

    def check_win(self):
        raise NotImplementedError("Subclasses must implement this method")
```

Figure 1: Generic Class

4 Game wise code logic :

In this section we would be looking at each game and checking how we implemented the logic uniquely for each game. The following are the set of games and their individual logic we used in this project:

Othello :

In this section we will be looking at our first game i.e. "Othello". The logic flow for the same is as follows:

- Firstly as we couldn't use for loops for the win condition we used a counter for the number of black and white markers which made it very easy for us to be able to decide the winner in case the game were to end by comparing those two quantities.

- We first capture the x and y coordinates from the :
  event.type == pygame.MOUSEBUTTONDOWN
  then we can find the clicked cell by x//8 and y//8 as the board is 8*8.

- The methods we used along with their uses are surmised as follows:

Methods	**Usage**
__init__	This is a dunder method used to initialize any game's players and the board with respect to Othello. We even initialized the center four marks as stated in the problem statement. We also had to define the dimensions of the board in this method.
switch	This is a simple method which is used to change the turn as the games work on turn basis logic where each player needs to switch the turn thus this function becomes significant.
drawgrid	This is a method which we use to draw the board it basically just prints the board .
drawmarks	This is a method which we use to draw the board it basically just prints the board as it is stored in the board array i.e. the marks are drawn.
move_isvalid	This is a very important function and as the name suggests tell if the move we have done is valid or not
makemove	This method from the cell position we recorded after checking that the move is valid put appropriate mark wrt the current player.
valid_possible	We also need to check for the conditions like where the player can to make a valid move then their turn must be skipped this function does just that. It goes through the whole board and if valid move is possible for any one of the cell then the game then the player has the option to make their move.
checkwin	It checks if the sum of the black and he white mark ar equal to 64 (8*8) i.e. if the whole board is filled or no. Also it checks if neither players have a valid move available then also it returns true i.e. the game is over.
isdraw	This method checks if the game is a draw . This scenario happens when the number of white marks = the number of black marks.
end_screen	this is just the ending screen showing all the possible options the player shave i.e. if they wish to play gain by continuing an going to the menu screen or quitting the whole program entirely.

Table 5: Methods used in Othello class

Connect 4:

In this section we will be looking at our second game i.e. "Connect 4". The logic flow for the same is as follows:

- Firstly as we couldn't use for loops for the win condition we used a counter for the number of red and white markers which made it very easy for us to be able to decide the winner in case the game were to end by comparing those two quantities.

- We first capture the x and y coordinates from the :
  event.type == pygame.MOUSEBUTTONDOWN
  then we can find the clicked cell by x//7 and y//7 as the board is 7*7.

- The methods we used along with their uses are surmised as follows:

Methods	**Usage**
__init__	This is a dunder method used to initialize any game's players and the board with respect to Connect 4. We also had to define the dimensions of the board in this method.Also we defined the had a array mn to define the y position where each subsequently added disk will go to.
switch	This is a simple method which is used to change the turn as the games work on turn basis logic where each player needs to switch the turn thus this function becomes significant.
drawgrid	This is a method which we use to draw the board it basically just prints the board .
drawmarks	This is a method which we use to draw the board it basically just prints the board as it is stored in the board array i.e. the disks are drawn.
move_isvalid	This is a very important function and as the name suggests tell if the move we have done is valid or not
makemove	This method from the cell position we recorded after checking that the move is valid put appropriate mark wrt the current playerand the possible row for that particular column depending ont he mn array.
checkwin	In this methd we basically iterate through the boar and loop for 4 disks of same color in a line.
isdraw	
end_screen	this is just the ending screen showing all the possible options the player shave i.e. if they wish to play gain by continuing an going to the menu screen or quitting the whole program entirely.

Table 6: Methods used in Connect 4 class

TicTacToe :

In this section we will be looking at our third and the final game i.e. "Tic Tac Toe". The logic flow for the same is as follows:

* Firstly as we couldn't use for loops for the win condition we used a counter for the number of black and white markers which made it very easy for us to be able to decide the winner in case the game were to end by comparing those two quantities.

* We first capture the x and y coordinates from the :
  event.type == pygame.MOUSEBUTTONDOWN
  then we can find the clicked cell by x//3 and y//3 as the board is 3*3.

* The methods we used along with their uses are surmised as follows:

Methods	**Usage**
__init__	This is a dunder method used to initialize any game's players and the board with respect to Othello.We also had to define the dimensions of the board in this method.
switch	Overrides the game classes switch method.
drawgrid	This is a method which we use to draw the board it basically just prints the board .
drawmarks	This is a method which we use to draw the board it basically just prints the board as it is stored in the board array i.e. the marks are drawn.
makemove	This method from the cell position we recorded after checking that the move is valid put appropriate mark wrt the current player.
checkwin	We used splicing and checked if the condition of 3 identical marks are in a single line.
isdraw	This method checks if all the cells have been covered or not.
end_screen	this is just the ending screen showing all the possible options the player shave i.e. if they wish to play gain by continuing an going to the menu screen or quitting the whole program entirely.

Table 7: Methods used in TickTackToe class

5 Leader board workflow :

In this section we would be explaining the file leader board.sh which contains the commands to display a table corresponding to each game showing how much wins,losses and wins/losses ratio is there for each player.

- First of all we have defined wins and losses as two 2D arrays which store where the first index is the games and the second holds the players.
- Then through seen array find out all the unique players in the game database.
- Then we iterate over seen and for each player find the wins ,losses and the Win/Losses ratio the Win/Losses ratio is defined by the following ratio:

  $ratio = (l == 0)?w : w/l$

- All three parameters are stored int he Data Associate array
- Then we input the result of the players request and sort wrt that.
- Then the required oclumn of the Data array is extracted and is sorted like:

  $asorti(key, idx, "@val_num_desc")$

- Finally using the idx array we print the data stored in the Data array.